Reviewer Pack — Minimal Rank of Quandle Representations vs Permutation Circu…

Entry 84d62441a02b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 10:08:18 UTC

Minimal Rank of Quandle Representations vs Permutation Circuit Depth for k-CLIQUE

Entry ID: `84d62441a02b` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-25 10:08:18 UTC

1. Conjecture

Field A (mathematical branch): Quandle Theory **Field B** (complexity object): Complexity Theory: Permutation Circuit Complexity (for k-CLIQUE)

Statement:

> ['For every k-clique instance, there exists a quandle representation of its incidence graph such that the minimal rank of this representation is lower bounded by k.', 'This lower bound holds for all instances with at most 40 vertices and is preserved under vertex permutations and subgraph isomorphisms.', 'If an instance with n vertices violates this bound, it can be used as a counterexample to refute the conjecture.']

Rationale (proposer's reasoning):

> ['Quandle theory is a branch of algebra that has been applied to various areas of mathematics but is rarely studied in complexity theory. The conjecture suggests that quandles can be used to provide a lower bound on the size of permutation circuits for k-clique, which could potentially lead to new insights into the complexity of k-clique.', 'The structure of a quandle can capture the symmetries and relationships within an incidence graph of a k-clique, providing a rich source of invariants that could be used to analyze the complexity of k-clique.']

Taxonomy category: `GCT_DET_PERM` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `2844652fb279a277`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if no generated quandle representation for any k-clique instance with n vertices (n ≤ 40) has a minimal rank below k. The conjecture is falsified if there exists at least one instance where the minimal rank of its quandle representation is less than k.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Quandle Theory" AND "permutation circuit complexity" AND k-CLIQUE" - "minimal rank" AND quandle representation AND "k-clique instance" - "vertex permutations" AND subgraph isomorphisms" AND lower bound" AND quandle

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def gaussian_elimination(matrix):
    rows, cols = len(matrix), len(matrix[0])
    for i in range(rows):
        max_row = i + max(range(i, rows), key=lambda x: abs(matrix[x][i]))
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
        for j in range(i + 1, rows):
            factor = Fraction(matrix[j][i], matrix[i][i])
            for k in range(cols):
                matrix[j][k] -= factor * matrix[i][k]
    return matrix

def rank_of_matrix(matrix):
    rows, cols = len(matrix), len(matrix[0])
    row_echelon_form = gaussian_elimination(matrix)
    rank = 0
    for i in range(rows):
        if any(row_echelon_form[i]):
            rank += 1
    return rank

def generate_k_clique(n, k):
    vertices = list(range(n))
    edges = []
    for i in range(k):
        for j in range(i + 1, k):
            edges.append((vertices[i], vertices[j]))
```

```
    for _ in range(n - k):
        u = random.choice(vertices)
        v = random.choice(vertices)
        if (u, v) not in edges and (v, u) not in edges:
            edges.append((u, v))
    return {u: [v for v in vertices if (u, v) in edges] for u in vertices}

def quandle_representation(graph):
    n = len(graph)
    q = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in graph[i]:
            if j in graph[i]:
                q[i][j] = 1
    return q

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    total_metric_value = 0
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        for _ in range(5):  # Ensure at least 5 instances per size
            graph = generate_k_clique(n, k=3)
            q = quandle_representation(graph)
            rank = rank_of_matrix(q)
            if rank < 3:
                conjecture_holds = False
                counterexample = f"n={n}, rank={rank}"
                break
            total_metric_value += rank
            instances_tested += 1

    return {
        "metric_name": "Minimal Rank",
        "metric_value": total_metric_value / instances_tested,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 2
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)
```

```
    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{r['counterexample']}\" first_failing_seed={fir
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_460868da.py", line 93, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_460868da.py", line 72, in run_trial
    rank = rank_of_matrix(q)
           ^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_460868da.py", line 31, in rank_of_matri
    row_echelon_form = gaussian_elimination(matrix)
                       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_460868da.py", line 24, in gaussian_elim
    factor = Fraction(matrix[j][i], matrix[i][i])
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   File "/usr/lib/python3.12/fractions.py", line 281, in __new__
    raise ZeroDivisionError('Fraction(%s, 0)' % numerator)
ZeroDivisionError: Fraction(0, 0)
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed before producing data, which means it did not complete its execution to verify the conjecture. | next: Re-run the test with appropriate error handling and ensure it completes successfully to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11204
2	propose	ollama_remote	glm4:latest	0	0	10863
3	preregistration	ollama_remote	glm4:latest	0	0	5934
4	novelty	ollama_remote	glm4:latest	0	0	4720
5	novelty	ollama_remote	glm4:latest	0	0	5613
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16351
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11089
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12892
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11017
10	judge	ollama_remote	glm4:latest	0	0	7934

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 97617 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/84d62441a02b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/84d62441a02b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/84d62441a02b.tar.gz` (if generated)